Application Retirement, Offboarding & Decommissioning (Microsoft Intune)

Implementation Solution Guide

Version 1.0.0 · 2026-07-20

Contents

Executive Summary 5

The problem . . . 5

The solution . . . 5

What you get . . . 5

Business impact . . . 5

Bottom line . . . 6

1. Solution Overview — Application Retirement, Offboarding & Decommissioning (Microsoft Intune) 6

What this solution is . . . 6

Why retirement needs its own process . . . 6

How it works — the retirement pipeline . . . 7

Benefits . . . 8

Limitations — read these before you start . . . 9

The standard deployment model . . . 9

Government (GCC / GCC High / DoD) note . . . 9

2. Requirements Guide — Application Retirement, Offboarding & Decommissioning 10

2.1 Intune role with app and assignment write access . . . 10

2.2 Microsoft Graph PowerShell SDK and read scopes . . . 11

2.3 The app must exist as an Intune app object with a WORKING uninstall command . . . 11

2.4 A reliable detection rule that proves presence and absence . . . 11

2.5 An install-base and usage evidence source . . . 11

2.6 A named owner and an approver for the decision . . . 12

2.7 Data-handling determination (the irreversibility check) . . . 12

2.8 Wave security groups (assigned, not dynamic) . . . 12

2.9 Knowledge of every artefact the app touches . . . 12

2.10 A supersedence/replacement readiness check (redundancy & EOL retirements only) . . . 13

Global Requirements — Intune Solutions (category-level) 13

G1. Microsoft Entra ID tenant . . . 13

G2. Microsoft Intune subscription . . . 13

G3. Administrative roles . . . 13

G4. Device enrollment in Intune . . . 13

G5. Pilot security group . . . 14

G6. Supported Windows baseline . . . 14

G7. Network access to Microsoft cloud endpoints . . . 14

3. Implementation Guide — Application Retirement, Offboarding & Decommissioning 14

Named objects (create with these EXACT names and reuse them verbatim everywhere) 14
Step 1 — Name the trigger and open the decision record 15
Step 2 — Capture the install base and usage evidence 15
Step 3 — Run the impact analysis 15
Step 4 — Build and communicate the plan and grace period 16
Step 5 — Create the wave groups (assigned) and the exclusion group 16
Step 6 — Protect data (the irreversibility checkpoint) 17
Step 7 — Point the app's assignment at Wave 0 with the Uninstall intent 17
Step 8 — (Replacement/migration retirements only) use supersedence instead 17
Step 9 — Validate Wave 0 18
Step 10 — Expand wave by wave 18
Step 11 — Reclaim licences and record the cost recovery 18
Step 12 — Handle stragglers (campaign-level, not per-installer) 19
Step 13 — Clean up every artefact 19
Step 14 — Produce the decommission evidence pack 20
Step 15 — Post-retirement monitoring 20
5. Testing & Validation Guide — Application Retirement 20
Gate 0 — Data preservation (before any uninstall) 20
Gate 1 — Assignment intent is correct (before Wave 0 runs) 21
Gate 2 — Pilot device uninstalled and detection is absent 21
Gate 3 — No collateral breakage 21
Gate 4 — Wave completion threshold 22
Gate 5 — Artefacts removed 22
Gate 6 — Evidence pack is complete and closure holds 22
Full-campaign regression check 22
6. Troubleshooting — Application Retirement (campaign-level) 23
"I assigned Uninstall but nothing is being removed" 23
"Some devices removed it, most didn't" (partial fleet removal) 23
"The app uninstalled, then came back" 23
"Detection still shows it installed after a successful uninstall" 24
"Deleting the app object didn't uninstall it" 24
"The Uninstall shim I created isn't detecting the existing install" 24
"Supersedence isn't removing the old app" 24
"The reports don't match what's on the devices" (reporting gaps) 25
Script / access problems 25
7. Best Practices — Application Retirement, Offboarding & Decommissioning 25

Decision discipline . . . 25
Protecting data (the irreversible step) . . . 26
Campaign mechanics . . . 26
Stragglers and reach . . . 26
Cleanup and proof . . . 26
Naming and repeatability . . . 27
Deliverables Summary 27
Documents . . . 27
PowerShell scripts . . . 28
How the pieces fit . . . 29
Appendix A — PowerShell Scripts 30

Executive Summary

The problem

Organisations are good at deploying applications and bad at retiring them. Apps that reached end-of-life, failed a security assessment, were made redundant by a merger, or sit installed on hundreds of devices that nobody uses — they linger, because removal feels risky and nobody owns the decision. The result is paid-for licences on unused software, unpatchable security exposure, and a tenant cluttered with orphaned groups, scripts, and app-control rules. When someone finally tries to remove an app, they either destroy user data they should have kept, or cancel the licence while the software is still 60% installed — often both.

The solution

This is a **repeatable retirement process plus tooling** for the end of the application lifecycle on an Intune-managed Windows fleet. It turns "delete the app and hope" into a governed campaign: a signed decision record, an impact analysis of what depends on the app, a communication plan and grace period, a hard **data-preservation checkpoint** before the irreversible step, a **staged uninstall campaign** (pilot → waves → broad) that respects how Intune's Uninstall intent actually behaves, disciplined straggler handling, cleanup of **every artefact** the app leaves behind, licence reclamation, and a **decommission evidence pack** that proves the app is genuinely gone. Five PowerShell scripts read the campaign state directly from Microsoft Graph and stay read-only by default.

What you get

- A **retirement pipeline** — trigger → decision → impact → comms → data → uninstall waves → stragglers → artefact cleanup → licence reclamation → evidence → monitoring — with an owner and an exit gate at every stage.
- The **truth about Intune uninstall**: how the Uninstall intent runs the app's stored uninstall command, why **Required beats Uninstall**, why deleting the app object removes nothing, and why the fleet's check-in rate bounds your completion rate.
- **Five PowerShell scripts** — install-base discovery, dependency/supersedence impact analysis, wave status/planning/straggler classification, leftover-artefact finder, and decommission evidence export.
- **Testing gates** that make an irreversible action safe, and **Best Practices** that prevent the classic data-loss and "nothing happened" failures.

Business impact

- **Real cost recovery:** licences reclaimed and reported, so the saving lands in a budget line.
- **Reduced risk exposure:** EOL and vulnerable apps are removed on a deadline, with proof.
- **No surprise data loss:** the data checkpoint stops the one-way door from destroying anything you must keep.

• **A clean tenant:** groups, scripts, app-control rules, and Conditional Access references go too — not just the app.

• **Audit-ready:** the evidence pack answers "is it gone?" with data, for security, compliance, and finance.

Bottom line

Every retired application is removed the way it should be — decided, communicated, data-protected, staged, cleaned up, licence-reclaimed, and proven gone — with a repeatable method and scripts your team keeps and reuses for the next app, at a fraction of the cost of ad-hoc removal or outside consulting.

1. Solution Overview — Application Retirement, Offboarding & Decommissioning (Microsoft Intune)

What this solution is

Every organisation is good at *deploying* applications and bad at *retiring* them. Apps accumulate: the vendor went end-of-life three years ago, two teams bought overlapping tools, a security finding landed on a product nobody owns, or you are paying maintenance on 400 licences for software that 26 people opened last quarter. Nobody removes it, because removal feels risky and nobody owns the decision.

This solution is the **end of the application lifecycle**: a repeatable, evidence-backed process to retire an application from a Microsoft Intune-managed Windows fleet and prove it is gone. It covers the decision (should this app be retired, and who says so), the impact analysis (who still uses it and what depends on it), the communication and grace period, the **staged uninstall campaign**, the data and licence obligations, the cleanup of everything the app leaves behind in your tenant, and the **evidence pack** that closes the ticket.

It is a **process and campaign** discipline, not a debugging manual. When an individual device's uninstall genuinely fails at the installer level — a broken MSI cache, a hung uninstall string, a 0x80070643 — that is deep Win32 uninstall failure remediation and belongs to a dedicated troubleshooting discipline. Here, a per-device failure is simply a **straggler**: this solution tells you how to count them, contain them, escalate them, and stop them from blocking the campaign.

Why retirement needs its own process

Deployment and retirement are not mirror images:

	Deployment	Retirement
Failure mode	User doesn't get the app	User loses access to something, sometimes with data
Reversibility	Re-deploy	Often **irreversible** — local data can be destroyed

	Deployment	Retirement
Who notices	The user, immediately	The user, sometimes weeks later, at the worst moment
Ends when	App is installed	App is gone **and** every artefact, licence, and reference is gone
Evidence needed	Install count	A defensible decommission record for audit, security, and finance

A retirement done badly produces the two worst outcomes at once: a user who lost data they needed, and an app that is still 60% installed while you have already cancelled the licence.

How it works — the retirement pipeline

The solution runs the standard **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model, shaped for removal:

```
Trigger → Decision Record → Impact Analysis → Comms + Grace Period
      → Data Handling → Uninstall Wave 0 (Pilot) → Validate
      → Waves 1..N (Broad) → Straggler Handling → Artefact Cleanup
      → Licence Reclamation → Evidence Pack → Post-Retirement Monitoring
```

Each stage has an owner, an input, and an exit gate. You do not advance a wave until the previous wave's gate passes — because unlike a deployment, you cannot un-remove an app.

The five retirement triggers

An app enters the retirement pipeline for one of five reasons. Naming the trigger matters, because it sets the urgency and who has to approve:

1. **End of life / end of support (EOL/EOS)** — the vendor has stopped shipping security fixes.

Planned, date-driven; the removal deadline is the vendor's EOS date.

1. **Security risk** — an unpatchable vulnerability, a failed vendor assessment, or a compliance

finding. Urgency is set by the security owner; grace periods shrink or vanish.

1. **Redundancy** — two or more tools do the same job after a merger, a re-org, or a platform

standardisation. The replacement app must exist and be deployed **first**.

1. **Low utilisation** — telemetry shows the install base dwarfs the active user base. Driven by

evidence, not opinion.

1. **Licence cost** — renewal is due and the spend is not justified. The removal must complete

before the renewal date, or you pay for another term.

How Intune actually removes an application

This is the mechanic the whole campaign rests on, and it is widely misunderstood:

• Intune apps carry an **assignment intent**: **Required**, **Available for enrolled devices**,

or **Uninstall**. The **Uninstall** intent is what removes software.

• For a **Win32 app**, the Uninstall intent causes the Intune Management Extension (IME) to run the app's stored **uninstall command line** — the one recorded when the app was created. If that command is wrong, missing, or interactive, the uninstall fails. The command is executed on the device; Intune does not "know how" to remove software on its own.

• For **line-of-business (MSI/LOB)** apps, Intune calls the MSI uninstall for the recorded product code.

• For **Microsoft Store apps**, the Uninstall intent removes the package for the targeted scope (device or user, depending on how it was published).

• **Intents conflict, and the strongest wins in a defined order.** If a device is in *both* a group assigned **Required** and a group assigned **Uninstall**, the app stays installed — **Required wins**. This is the single most common reason a retirement campaign appears to do nothing. You must **remove the old Required assignment**, not merely add an Uninstall assignment on top.

• **Deleting the app object from Intune does NOT uninstall it from devices.** It only stops Intune managing it, and it destroys your ability to target the app for removal or to report on it. Deleting the app is the **last** step of the campaign, never the first.

• **Supersedence with "Uninstall previous version"** is a different mechanism — it removes an app as a side effect of installing a replacement. It is the right tool when retirement means *migration* to a successor app, and it is covered in this solution as a retirement path.

• The uninstall only happens **when the device checks in** and the IME evaluates the app workload. A device that never checks in never uninstalls anything. Your campaign's real completion rate is bounded by your fleet's check-in rate.

Waves, not a big bang

The campaign targets **assigned Entra security groups**, one per wave, so that at any moment the blast radius is a number you chose:

• **Wave 0 — Pilot:** 5–15 devices whose owners know it is happening and have agreed to be first.

• **Wave 1 — Early:** roughly 10% of the install base, spread across departments and device types.

• **Wave 2..N — Broad:** the remainder, in tranches you can support.

• **Retain / Exclude:** devices with a documented, time-limited exemption. This group is an **exclusion** on every wave assignment, and it must have an expiry date or it becomes permanent.

Benefits

• **A decision you can defend.** Every retirement carries a written decision record with the trigger, the evidence, the approver, and the replacement.

• **No surprise data loss.** Data obligations are identified and satisfied *before* the destructive step, with an explicit go/no-go checkpoint.

• **Controlled blast radius.** Waves, exclusions, and gates mean a mistake affects ten devices, not

ten thousand.

- **Real cost recovery.** Licences are reclaimed and reported, so the saving shows up in a budget line instead of being assumed.
- **A genuinely clean tenant.** Groups, assignments, scripts, app-control rules, and Conditional Access references are removed too — not just the app.
- **Audit-ready proof.** The evidence pack answers "is it gone?" with data, not with an opinion.

Limitations — read these before you start

- **Removal can be irreversible and can destroy local data.** Many uninstallers delete `%ProgramData%` and `%LOCALAPPDATA%` content, licence activations, local databases, cached documents, and plug-in configurations. Some do not. You must determine which, per app, before Wave 0. This solution puts a hard checkpoint there.
- **Intune cannot uninstall what it did not install, unless you give it a command.** For software installed by SCCM/Configuration Manager, a legacy script, or a user, you must create an Intune app object (or a remediation) with a working uninstall command to target it.
- **You cannot force a device to check in.** Offline, decommissioned, and long-idle devices will not uninstall. The campaign must define what "done" means in the presence of unreachable devices.
- **Per-device uninstall failures are out of scope here.** When a specific machine's uninstall genuinely errors at the installer level, escalate to a Win32 uninstall failure remediation discipline; this solution covers detecting, counting, containing and escalating those stragglers.
- **Intune does not report "usage".** Install state is not utilisation. Real usage evidence comes from the vendor's admin console, licence-server logs, SSO/Entra sign-in logs for the app's enterprise application, or an endpoint analytics/telemetry source. This solution shows you where to get it; it cannot manufacture it.
- **Microsoft Store (new) app uninstall behaviour differs by publish scope** (user vs device). Confirm the scope before assuming an Uninstall assignment will reach the user's copy.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — applied as: assess the install base and impact, configure the wave groups and Uninstall assignments, pilot on Wave 0, validate with the testing gates, then expand wave by wave to the whole install base, closing with cleanup and evidence.

Government (GCC / GCC High / DoD) note

The retirement mechanics — assignment intents, the Uninstall intent, supersedence, wave targeting, artefact cleanup — are **the same** in US Government clouds. What differs is **where you sign in and which service endpoints you talk to**. Wherever this guide names a portal or Graph endpoint, use the sovereign equivalent:

Commercial	US Gov (GCC High / DoD)
`https://intune.microsoft.com`	`https://portal.azure.us` (and, where enabled, `https://intune.microsoft.us`)
`https://entra.microsoft.com`	`https://portal.azure.us`
`https://login.microsoftonline.com`	`https://login.microsoftonline.us`
`https://portal.manage.microsoft.com` (Company Portal web)	`https://portal.manage.microsoft.us`
`manage.microsoft.com` (service endpoint)	`manage.microsoft.us`
`https://graph.microsoft.com`	`https://graph.microsoft.us` (GCC High) · `https://dod-graph.microsoft.us` (DoD)

For the PowerShell scripts, connect with `Connect-MgGraph -Environment USGov` (GCC High) or `-Environment USGovDoD` (DoD) — every script in this package accepts an `-Environment` parameter for exactly this. Feature availability in sovereign clouds can lag Commercial; where a specific control in this guide is not present in your tenant, **verify for GCC High** against Microsoft's current US Government service documentation before assuming it is a fault.

2. Requirements Guide — Application Retirement, Offboarding & Decommissioning

Verify **every** requirement before you begin a retirement campaign. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

Throughout this solution we use a **worked example** so every rule and object is concrete. We are retiring a fictional line-of-business app:

- **App display name (as it appears in Intune):** `LegacyTrack Client`
- **Installer type:** Windows Installer (MSI), `LegacyTrackSetup.msi`, MSI product code `{2F1A9C44-7B3E-4E22-9C51-1A2B3C4D5E6F}`
- **Vendor status:** end of support reached; replaced by a SaaS product, so this is an **EOL/EOS + redundancy** retirement.
- **Retirement code:** `APPRET-LGT` (a short token you invent per campaign; used in every object name below so a whole campaign's objects sort together and are easy to clean up).

Replace the names, code, and product code with your own app's values.

2.1 Intune role with app and assignment write access

Description: An operator account holding **Intune Administrator** (or a custom role with **Managed apps** and **Assignment** read/write). **Why:** The campaign edits app assignments (adds Uninstall, removes Required), edits groups, and finally deletes the app object. **Verify:** Entra → **Roles and administrators** →

confirm **Intune Administrator** on your account. **Implement:** Assign the role (G3). Reserve Global Administrator for the app-object deletion step only if your custom role cannot delete apps.

2.2 Microsoft Graph PowerShell SDK and read scopes

Description: The **Microsoft Graph PowerShell SDK** installed, with consent to the read scopes `DeviceManagementApps.Read.All`, `DeviceManagementManagedDevices.Read.All`, `DeviceManagementConfiguration.Read.All`, and `Group.Read.All`. **Why:** The five scripts in this package read install base, assignments, dependencies, group membership, and leftover artefacts from Graph. Read-only scopes are enough for discovery, reporting, and evidence; the two gated scripts request write scopes only when you confirm an action. **Verify:** In PowerShell run `Get-Module -ListAvailable Microsoft.Graph.Authentication`; run `Connect-MgGraph -Scopes 'DeviceManagementApps.Read.All'` and confirm it connects. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. On first connect, a Global Administrator may need to grant admin consent for the scopes.

2.3 The app must exist as an Intune app object with a WORKING uninstall command

Description: The app you are retiring is present in Intune → **Apps**, and — critically — its **uninstall command line** (Win32) or **product code** (MSI/LOB) is correct and silent. **Why:** The **Uninstall** intent runs *this stored command* on the device. If the app was created years ago with a wrong or interactive uninstall string, every uninstall will fail. If the software was installed **outside Intune** (SCCM, a script, a user), there is no app object to target — you must create one. **Verify:** Intune → **Apps** → **All apps** → open `LegacyTrack Client` → **Properties** → **Program** and read the **Uninstall command**. For the worked example it must be `msiexec /x {2F1A9C44-7B3E-4E22-9C51-1A2B3C4D5E6F} /qn /norestart`. Test that command manually on one clean device and confirm exit code `0`, `1605`, or `3010`. **Implement:** If the command is wrong, correct it in the app's **Program** tab before starting. If no Intune app object exists for software you need to remove, create a **Windows app (Win32)** whose package is a tiny placeholder and whose **uninstall command** is the app's real silent uninstall string, plus a **detection rule** that matches the installed app (so Intune can report it as present, then absent). Name it `APPRET-LGT – LegacyTrack Client – Uninstall Shim`.

2.4 A reliable detection rule that proves presence and absence

Description: A deterministic detection rule (MSI product code, a versioned file, or a registry value) that is TRUE when the app is installed and FALSE when it is gone. **Why:** The Uninstall intent's success/failure reporting, and your evidence pack, both depend on detection flipping from present to absent. A guessed rule produces false "removed" or false "failed" results. **Verify:** Intune → app → **Properties** → **Detection rules**. For the worked example: **MSI** rule on product code `{2F1A9C44-7B3E-4E22-9C51-1A2B3C4D5E6F}`. **Implement:** Capture the app's MSI product code, or an installed file + version (e.g. `C:\Program Files\LegacyTrack\lt.exe`, version `8.4.0`), or its uninstall registry key, and set exactly one authoritative detection rule.

2.5 An install-base and usage evidence source

Description: Two distinct facts before you retire: **how many devices have the app** (install base) and **whether anyone still uses it** (utilisation). **Why:** Install base bounds the campaign size and comes from Intune. Utilisation justifies the retirement and does **not** come from Intune — install state is not usage. You need a real usage signal to defend a "low utilisation" trigger and to find active users who need warning. **Verify:** Run `APPRET-Discovery.ps1` for the install base. For usage, confirm you can obtain at least one of: the vendor/admin console's active-user report, licence-server or activation logs, the app's **Enterprise application** sign-in logs in Entra (if it uses SSO), or an endpoint analytics/telemetry export. **Implement:** If you have no usage signal at all, treat the retirement as **evidence-incomplete**: extend the grace period, deploy a "this app is being retired" notice, and use the grace period itself as the usage probe (who complains = who uses it).

2.6 A named owner and an approver for the decision

Description: A business owner who is accountable for the app, and an approver empowered to sign off its removal. **Why:** Retirement is a decision, not a task. Removing an app nobody will vouch for causes the outage that stops the next campaign getting approved. **Verify:** The decision record (see Implementation Step 2) has a named owner and approver, both real people, before Wave 0. **Implement:** If no owner can be found, that *is* evidence for retirement — but the approver must then be a senior IT/security stakeholder who explicitly accepts the risk in writing.

2.7 Data-handling determination (the irreversibility check)

Description: A per-app finding of **what the uninstall destroys** and **what data must be preserved** before removal (exports, licence keys, local databases, user documents, config). **Why:** This is the one-way door. Many uninstallers wipe `%ProgramData%\<vendor>` and per-user app data. If data lives only there and is under a retention obligation, you must export it first. **Verify:** On a test device, install the app, create representative data, run the uninstall command, and inspect what survives under `C:\ProgramData\`, `%LOCALAPPDATA%`, `%APPDATA%`, and the registry. Record the result. **Implement:** If the uninstall destroys data you must keep, script or perform the export **before** Wave 0 and record it in the evidence pack. If retention is uncertain, get a written retention decision from data/records owners — never guess.

2.8 Wave security groups (assigned, not dynamic)

Description: Entra **security groups**, membership type **Assigned**, one per wave, plus one exclusion group. **Why:** A retirement must have a blast radius you chose. **Assigned** groups give you exact, auditable control over which devices uninstall this cycle; a dynamic rule could sweep the whole fleet in one sync. The exclusion group protects documented exemptions. **Verify:** Entra → **Groups**; confirm the wave groups exist as **Assigned** security groups (see Implementation Step 5 for the exact names). **Implement:** Create them per Step 5. Populate device membership from the `APPRET-Discovery.ps1` install-base export.

2.9 Knowledge of every artefact the app touches

Description: An inventory of the tenant objects that reference this app: its assignments, dependency/supersedence links, the Entra groups used to target it, any **detection or requirement scripts**, **WDAC / App Control for Business** rules, **Conditional Access** policies that name the app's enterprise application, and any **Configuration Manager (SCCM)** deployment if the app was co-managed. **Why:** Deleting the app but leaving its groups, app-control rules, and CA references behind is not decommissioning — it is hiding the evidence. Orphaned app-control rules in particular can silently block or

allow things long after the app is gone. **Verify:** Run `APPRET-ArtifactFinder.ps1`; it lists objects that reference the app or its target groups. **Implement:** Build the artefact list before Wave 0 so cleanup at the end is a checklist, not an archaeology dig.

2.10 A supersedence/replacement readiness check (redundancy & EOL retirements only)

Description: When retirement means migrating to a **replacement app**, that replacement is already packaged, deployed, and validated on the target devices. **Why:** If you remove the old app before the new one is proven, you create an outage. Supersedence with **Uninstall previous version = Yes** ties the two together so a device is never left with neither. **Verify:** Intune → **Apps**; the replacement app (e.g. the SaaS client `LegacyTrack Cloud Agent`) reports **Installed** on the pilot devices. **Implement:** Deploy and validate the replacement first (a normal app-deployment exercise), then configure supersedence in Implementation Step 8.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Application Retirement, Offboarding & Decommissioning

Follow these steps **in order**. Nearly everything is in the **Microsoft Intune admin center** (`https://intune.microsoft.com`) and the **Microsoft Entra admin center** (`https://entra.microsoft.com`). *(US Gov: sign in at `https://portal.azure.us`; substitute the sovereign endpoints from the Solution Overview table throughout.)*

> ***Model:*** *Decide → analyse impact → communicate → protect data → uninstall Wave 0 (pilot) → validate → expand wave by wave → handle stragglers → clean up every artefact → reclaim licences → produce the evidence pack → monitor.* ***You cannot un-remove an app****, so no wave advances until the prior wave's gate passes, and the data checkpoint (Step 6) is a hard go/no-go.*

Named objects (create with these EXACT names and reuse them verbatim everywhere)

We use the `APPRET-<code> – <App> – <Purpose> – <Stage>` convention so every object created for one retirement sorts and cleans up together. For the worked example (`LegacyTrack Client`, code `APPRET-LGT`):

Object	Name to create
Pilot wave group (Wave 0)	`APPRET-LGT – LegacyTrack Client – Uninstall – Wave 0 Pilot`

Object	Name to create
Early wave group (Wave 1)	`APPRET-LGT – LegacyTrack Client – Uninstall – Wave 1 Early`
Broad wave group (Wave 2)	`APPRET-LGT – LegacyTrack Client – Uninstall – Wave 2 Broad`
Exemption / exclusion group	`APPRET-LGT – LegacyTrack Client – Retain – Exempt (expires 2026-09-30)`
Uninstall shim app (only if no app object exists)	`APPRET-LGT – LegacyTrack Client – Uninstall Shim`
Retirement notice (proactive remediation, optional)	`APPRET-LGT – LegacyTrack Client – Retirement Notice`
The decision record document	`APPRET-LGT – LegacyTrack Client – Decision Record`

Step 1 — Name the trigger and open the decision record

1. Identify which of the five triggers applies (EOL/EOS, security risk, redundancy, low utilisation, licence cost). Write it down — it sets urgency and approver.
1. Create the decision-record document `APPRET-LGT – LegacyTrack Client – Decision Record` (a controlled doc/wiki page — content, not an Intune object). It will collect: trigger, evidence, named owner, approver + sign-off date, replacement app (if any), data determination, planned wave dates, and the licence/renewal deadline. You fill it in as you go; it becomes the front page of the evidence pack.

Step 2 — Capture the install base and usage evidence

1. Run `APPRET-Discovery.ps1 -AppName "LegacyTrack Client"`. It reports the app's Intune object, assignment intents and target groups, dependency/supersedence links, and the **per-device install list** (device name, primary user, last check-in). Save the CSV — it is your wave-planning source and evidence exhibit A.
1. Obtain the **usage** signal (Requirement 2.5) and attach it to the decision record. Install base (from Intune) and utilisation (from the vendor/SSO/telemetry) are two different numbers; the gap between them is your retirement justification.
1. **Gate 1 — Decision:** the owner and approver sign the decision record. Do not proceed without it.

Step 3 — Run the impact analysis

1. Run `APPRET-ImpactAnalyzer.ps1 -AppName "LegacyTrack Client"`. It surfaces, from Graph:

• devices/users that still have the app (who is affected),

• **dependencies** — other Intune apps that declare this app as a dependency (removing it breaks them),

• **supersedence chains** the app sits in (is it superseding, or superseded by, something?),

• the **Entra groups** used to assign it, and their membership counts,

• Win32 apps that share the same detection/requirement **script** (shared scripts are a cleanup hazard).

1. Review the report with the owner. Anything the app is a **dependency for** must be resolved first (re-point or retire the dependent app too) — otherwise the dependent app will try to reinstall `LegacyTrack Client` and fight your Uninstall intent.

1. Record the affected-user count and any dependency findings in the decision record.

Step 4 — Build and communicate the plan and grace period

1. Decide the grace period from the trigger: security = short or none; EOL/redundancy/cost = a stated window (commonly 2–4 weeks) with a hard removal date tied to the vendor EOS or the licence renewal.

1. Draft the user communication: what is being removed, why, the replacement (if any), the removal date, what the user must do (save/export anything they need), and the support contact. Send it to the affected users identified in Step 3.

1. *(Optional but recommended)* Deploy the notice **on the device** as an Intune **proactive remediation **named** `APPRET-LGT – LegacyTrack Client – Retirement Notice` **(Intune** → Devices → Scripts and remediations → Remediations → Create**), scoped to the install-base group, that shows a toast/notification. This reaches users who ignore email. Keep it detection-only (no destructive action).

1. Record the comms date and grace-period end in the decision record.

Step 5 — Create the wave groups (assigned) and the exclusion group

1. Entra → **Groups** → **New group** → **Group type: Security** → **Membership type: Assigned** → name `APPRET-LGT – LegacyTrack Client – Uninstall – Wave 0 Pilot` → **Create**. Add 5–15 pilot devices (owners who agreed to be first) as members.

1. Repeat to create `APPRET-LGT – LegacyTrack Client – Uninstall – Wave 1 Early` and `APPRET-LGT – LegacyTrack Client – Uninstall – Wave 2 Broad`. Leave their membership empty for now; you will populate them from the install-base CSV as each wave opens.

1. Create the exclusion group `APPRET-LGT – LegacyTrack Client – Retain – Exempt (expires 2026-09-30)` → **Assigned** security group → add only devices with a **documented, dated** exemption. The expiry date is in the name so an audit sees when the exemption lapses.

1. *(Optional)* Use `APPRET-WaveReporter.ps1 -Action Plan` to split the install-base CSV into balanced

wave membership lists (spread across departments and device types), then add members accordingly.

Step 6 — Protect data (the irreversibility checkpoint)

1. Confirm the data determination from Requirement 2.7: does the uninstall destroy data you must keep?

2. If **yes**, complete the export **now**, before any uninstall assignment exists:

• perform or script the export (vendor export tool, copy of the local database/config, licence-key

capture), store it in your retention location, and

• record the export location, date, and operator in the decision record.

1. Capture the licence/entitlement state now too (see Step 11) — some products deactivate the licence

on uninstall, and you want the seat count *before* removal.

1. **Gate 2 — Data go/no-go:** the owner confirms in writing that either (a) the uninstall destroys

nothing that must be kept, or (b) all required data has been exported and verified. **Do not create any Uninstall assignment until this gate passes.** This is the one-way door.

Step 7 — Point the app's assignment at Wave 0 with the Uninstall intent

1. Intune → **Apps** → **All apps** → open `LegacyTrack Client` → **Properties** → **Assignments** → **Edit**.

2. **Remove the existing install assignment** that keeps the app on pilot devices. Under **Required**

(and **Available for enrolled devices**), remove any group whose membership overlaps Wave 0.

> ***Critical:*** *the **Required** intent beats **Uninstall**. If a Wave 0 device is still in a group assigned Required, it will **not** uninstall. Removing the old install assignment for those devices is mandatory, not optional.*

1. Under the **Uninstall** intent → **Add group** → select

`APPRET-LGT – LegacyTrack Client – Uninstall – Wave 0 Pilot`.

1. Under the **Uninstall** intent's **Excluded groups** (or as a global exclusion) → add

`APPRET-LGT – LegacyTrack Client – Retain – Exempt (expires 2026-09-30)`.

1. **Review + Save.**

Step 8 — (Replacement/migration retirements only) use supersedence instead

If retirement means migrating to a successor app, prefer supersedence over a bare Uninstall so no device is ever left with neither app:

1. Open the **replacement** app (e.g. `LegacyTrack Cloud Agent`) → **Properties → Supersedence →

Edit → Add**.

1. Select the retiring app `LegacyTrack Client` → set **Uninstall the previous version = Yes**.

2. **Review + Save.** As the replacement deploys to a wave, Intune removes `LegacyTrack Client` on the

same device. You still drive the rollout wave by wave via the replacement app's assignment groups, and you still run the same validation gates.

Step 9 — Validate Wave 0

1. On a pilot device, sync policy (**Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**), or

wait a check-in cycle.

1. Run `APPRET-WaveReporter.ps1 -AppName "LegacyTrack Client" -Action Status`. Confirm Wave 0 devices

move to **Uninstalled / Not installed** and detection reports the app **absent**.

1. Complete the **Testing Guide** Wave-0 gates (uninstalled, detection absent, no collateral breakage,

data preserved).

1. **Gate 3 — Pilot pass:** every reachable Wave 0 device is confirmed uninstalled and clean, with no

dependency fallout. Investigate any device still showing installed (Step 12) before expanding.

Step 10 — Expand wave by wave

1. Only after Gate 3 passes, populate `APPRET-LGT – LegacyTrack Client – Uninstall – Wave 1 Early`

from the install-base CSV (~10%), then add it under the app's **Uninstall** intent (Step 7.3). **Save.**

1. Let it run a check-in cycle; re-run `APPRET-WaveReporter.ps1 -Action Status`; clear stragglers

(Step 12). When Wave 1 meets the completion threshold, open **Wave 2 Broad** the same way.

1. Continue until every non-exempt device in the install base has been targeted and reported.

2. Track the **residual installed count** every cycle. "Done enough to proceed to cleanup" is a number

you set in the decision record (e.g. ≥98% of reachable devices uninstalled, with all remaining devices accounted for as offline/decommissioned/exempt).

Step 11 — Reclaim licences and record the cost recovery

1. With the fleet uninstalled, reclaim entitlements: in the **vendor/licence console** release or

downgrade the freed seats; if the app was licence-server based, confirm the freed check-outs.

1. If the app was assigned via an **Entra enterprise application** (SSO/app licences), remove the app

assignment/licences there too.

1. Record **seats before vs after**, the per-seat cost, and the annualised saving in the decision record. This is the number finance keeps.
1. **Cancel the renewal** before its date if the trigger was cost/EOL — capture the confirmation.

Step 12 — Handle stragglers (campaign-level, not per-installer)

A "straggler" is a device that has not uninstalled. Classify each with `APPRET-WaveReporter.ps1 -Action Stragglers`:

• **Never checked in / offline:** the IME never ran. Chase the device (is it retired? re-imaged? lost?). If genuinely decommissioned, move it to the exempt/decommissioned group and record it — do not let it hold the campaign open forever.

• **Checked in, still installed, Required somewhere:** an install assignment is still winning (Step 7.2). Find and remove it — `APPRET-ArtifactFinder.ps1` shows which group/assignment.

• **Checked in, uninstall reported Failed:** the installer itself errored (bad uninstall string, 0x80070643, hung MSI). **This is where you escalate to deep Win32 uninstall failure remediation** — it is a per-device installer problem, not a campaign problem. Record the device and hand it to that discipline; do not block the wave on it. Keep a live straggler list in the evidence pack with each device's class and disposition.

Step 13 — Clean up every artefact

Now, and only now, remove what the app leaves behind. Use `APPRET-ArtifactFinder.ps1` as the checklist:

1. **Uninstall assignments & the app object:** once the residual count is accepted, remove the Uninstall assignments, then Intune → app → **…** → **Delete** to delete the app object. *(Deleting earlier would have stopped you targeting or reporting on it — that is why it is last.)*
1. **Wave & exempt groups:** delete the `APPRET-LGT – … – Uninstall – Wave *` groups and the `Retain – Exempt` group once no longer referenced.
1. **Detection / requirement scripts:** remove scripts that only this app used (Intune → **Apps** → the app's rules were deleted with it; check **Devices** → **Scripts and remediations** for standalone scripts named for the app). Do **not** delete a script another live app shares (the ImpactAnalyzer flagged shared scripts).
1. **WDAC / App Control for Business rules:** remove allow/deny rules that named the retired app's binaries or publisher. Orphaned app-control rules are a real hazard — a stale allow rule widens your attack surface; a stale deny rule blocks a future app with the same name.
1. **Conditional Access references:** Entra → **Protection** → **Conditional Access** → find any policy that targeted the app's **enterprise application**; remove the app from it or retire the policy. Then remove the **enterprise application / app registration** if nothing else uses it.
1. **SCCM / Configuration Manager remnants:** if the app was co-managed, retire its ConfigMgr

application/deployment and collections so it cannot redeploy. *(Vendor-console specifics — verify in your ConfigMgr documentation.)*

1. **The Uninstall Shim app** (if you created one in Requirement 2.3): delete it once the fleet is clean.

1. Tick each removed artefact in the decision record.

Step 14 — Produce the decommission evidence pack

1. Run `APPRET-EvidenceExport.ps1 -AppName "LegacyTrack Client" -OutFolder .\APPRET-LGT-Evidence`.

It compiles: the final install/uninstall status, the residual-device list with dispositions, the artefact-cleanup confirmation (app object gone, groups gone, no CA/app-control references remain), the data-export record reference, and the licence-reclamation numbers.

1. Attach the decision record (Step 1) as the front page. This pack is **proof of decommission** — the answer to "is `LegacyTrack Client` genuinely gone?"

Step 15 — Post-retirement monitoring

1. For a defined window (commonly **30–60 days**), re-run `APPRET-WaveReporter.ps1 -Action Status` and `APPRET-ArtifactFinder.ps1` weekly to catch **reappearance**: an offline device that came back and re-checked-in still installed, a re-imaged machine that got the app from an old task sequence, or a user who reinstalled it manually.

1. Any reappearance → add the device to the next Uninstall wave, or find and kill the source (a lingering SCCM deployment, an old image, a Required assignment you missed).

1. When two consecutive weekly checks show zero installed and zero artefacts, close the campaign.

Order of operations

`Name trigger + open decision record → Discovery (install base + usage) → Impact analysis → Comms + grace period → Wave groups + exclusion → PROTECT DATA (gate) → Uninstall→Wave 0 (or supersedence) → Validate (gate) → Expand waves → Reclaim licences → Stragglers → Artefact cleanup → Evidence pack → Monitor.`

5. Testing & Validation Guide — Application Retirement

Retirement is irreversible, so validation is not a formality — each gate must **pass** before the next wave. Use the worked example `LegacyTrack Client` (code `APPRET-LGT`). Record every result in the decision record / evidence pack.

Gate 0 — Data preservation (before any uninstall)

Purpose: prove the one-way door is safe to open.

1. On a test device, install `LegacyTrack Client`, create representative data, then run the uninstall command manually.
1. Inspect `C:\ProgramData\`, `%LOCALAPPDATA%`, `%APPDATA%`, and the app's registry keys for what survives.
1. **PASS:** either the uninstall destroys nothing that must be kept, OR all required data has been exported and the export verified (open one exported file and confirm it is intact).
1. **FAIL:** required data is destroyed and not yet exported — stop; complete Step 6 export first.

Gate 1 — Assignment intent is correct (before Wave 0 runs)

Purpose: prove the Uninstall intent will actually win.

1. Run `APPRET-Discovery.ps1 -AppName "LegacyTrack Client"`.
2. **PASS:** the app shows an **Uninstall** assignment targeting `APPRET-LGT – LegacyTrack Client – Uninstall – Wave 0 Pilot`, and **no Required/Available** assignment overlaps the same pilot devices, and the exempt group is excluded.
1. **FAIL:** a Required assignment still covers a pilot device (Required beats Uninstall) — remove it (Implementation Step 7.2) and re-check.

Gate 2 — Pilot device uninstalled and detection is absent

Purpose: prove removal on a real device.

1. Sync a Wave 0 device; wait a check-in cycle.
2. Run `APPRET-WaveReporter.ps1 -AppName "LegacyTrack Client" -Action Status`.
3. On the device, confirm the app is gone (Start menu, `Program Files`, `appwiz.cpl`).
4. **PASS:** the device reports **Uninstalled / Not installed**, detection reports the app **absent**, AND the exported/checkpoint data (Gate 0) is still intact where you stored it.
1. **FAIL:** still installed → Gate 1 issue or a per-device installer failure (Troubleshooting).

Gate 3 — No collateral breakage

Purpose: prove nothing that depended on the app broke.

1. Review the `APPRET-ImpactAnalyzer.ps1` dependency findings against the pilot devices.
2. On a pilot device, exercise any app or workflow that the ImpactAnalyzer flagged as dependent.
3. **PASS:** no dependent app tried to reinstall `LegacyTrack Client`; no user-facing workflow broke; the replacement app (if this is a migration) works.
1. **FAIL:** a dependent app is fighting the uninstall (it re-installs the app) → resolve the

dependency (re-point or retire the dependent app) before expanding.

Gate 4 — Wave completion threshold

Purpose: prove a wave is "done enough" to open the next.

1. After each wave's check-in cycle, run `APPRET-WaveReporter.ps1 -Action Status`.
2. **PASS:** the wave meets the completion threshold set in the decision record (e.g. ≥98% of *reachable* devices uninstalled) and every non-removed device is accounted for by `APPRET-WaveReporter.ps1 -Action Stragglers` (offline / decommissioned / escalated / exempt).
1. **FAIL:** unclassified installed devices remain → run Stragglers and disposition each before opening the next wave.

Gate 5 — Artefacts removed

Purpose: prove decommission, not just uninstall.

1. Run `APPRET-ArtifactFinder.ps1 -AppName "LegacyTrack Client" -AppCode "APPRET-LGT"`.
2. **PASS:** the app object is deleted, the `APPRET-LGT` wave/exempt groups are deleted (or flagged SAFE and then deleted), no Conditional Access policy references the app's enterprise application, no enterprise application / app registration remains, and app-specific scripts are removed. WDAC / App Control and SCCM remnants confirmed removed manually.
1. **FAIL:** any artefact still references the app → clean it (Implementation Step 13) and re-run.

Gate 6 — Evidence pack is complete and closure holds

Purpose: prove it, and prove it stays gone.

1. Run `APPRET-EvidenceExport.ps1 -AppName "LegacyTrack Client" -OutFolder .\APPRET-LGT-Evidence`.
2. **PASS:** `summary.txt` shows the app object NOT FOUND, residual installed accounted for, and remaining artefacts = 0; the data-export record and licence-reclamation numbers are attached; and two consecutive weekly post-retirement checks (Implementation Step 15) show zero installed and zero artefacts.
1. **FAIL:** the app reappears on a device, or an artefact returns → find the source (old image, lingering SCCM deployment, a missed Required assignment) and remediate before closing.

Full-campaign regression check

Before declaring the campaign closed, walk Gates 0→6 once end to end for the whole install base — not just the last wave. A retirement is complete only when the app is uninstalled fleet-wide, every straggler is dispositioned, every artefact is gone, the evidence pack is compiled, and monitoring is clean for the defined window.

6. Troubleshooting — Application Retirement (campaign-level)

This section covers **campaign-level** problems — the removal not progressing across the fleet as a whole. For a **single device whose uninstall genuinely errors at the installer level** (a bad uninstall string, `0x80070643`, a hung MSI, a corrupt product cache), that is deep Win32 uninstall failure remediation: record the device with `APPRET-WaveReporter.ps1 -Action Stragglers`, hand it to that discipline, and keep the wave moving. Do not stall the campaign on one machine.

Worked example throughout: `LegacyTrack Client`, code `APPRET-LGT`.

"I assigned Uninstall but nothing is being removed"

Most common cause — Required beats Uninstall. A device still in a group assigned **Required** (or **Available** and installed) will not uninstall while that assignment exists.

- Run `APPRET-Discovery.ps1 -AppName "LegacyTrack Client"` and read every assignment intent.
- Remove the old **Required/Available** assignment for the devices you are retiring (Implementation Step 7.2). Adding an Uninstall on top is not enough.
- `APPRET-ArtifactFinder.ps1` shows which group still carries the winning intent.

Second cause — the device hasn't checked in. The Uninstall only runs when the IME evaluates the app on check-in. Force a sync (**Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**) or wait a cycle. See "partial fleet removal" below.

Third cause — wrong target. Confirm the Wave 0 group actually contains the pilot devices (device objects, not user objects, for a device-targeted app), and that those devices are members *now* (group membership changes also take a sync to apply).

"Some devices removed it, most didn't" (partial fleet removal)

This is normal and expected — completion is bounded by check-in.

- Run `APPRET-WaveReporter.ps1 -Action Status` to see the split, then `-Action Stragglers` to classify the remainder.
- **Offline / never checked in:** the IME never ran. Chase the device — is it powered off, off the network, re-imaged, or decommissioned? Genuinely dead devices go to the decommissioned/exempt group and are recorded; they must not hold the campaign open.
- **Checked in but still installed:** almost always a winning Required assignment (see above).
- Set a **completion threshold on reachable devices** in the decision record; don't chase 100% of a fleet that includes dead hardware.

"The app uninstalled, then came back"

Something is re-deploying it.

- **A dependent app:** another Intune app declares `LegacyTrack Client` as a dependency with auto-install, so it reinstalls after removal. `APPRET-ImpactAnalyzer.ps1` section [1] lists dependents — resolve them first (re-point or retire the dependent app).
- **A lingering SCCM/ConfigMgr deployment:** if the app was co-managed, a ConfigMgr deployment can reinstall it. Retire that deployment (Implementation Step 13.6).
- **An old image / task sequence:** a re-imaged device gets the app from a stale build. Update the image or add the device to the next Uninstall wave.
- **A user reinstalled it manually:** remove the installer's availability (Company Portal "Available" assignment) and communicate.

"Detection still shows it installed after a successful uninstall"

- The **detection rule is wrong** — it is matching something the uninstall didn't remove (a leftover folder, a registry key the uninstaller leaves). Verify the rule (Requirement 2.4) points at a value that truly disappears on uninstall (MSI product code is the most reliable).
- Or the device simply hasn't re-reported yet; force a sync and re-check next cycle.

"Deleting the app object didn't uninstall it"

Correct — it never will. Deleting the app object only stops Intune managing and reporting it; the software stays on every device, now invisible to you. If you deleted it too early, **recreate the app object** (or an uninstall shim, Requirement 2.3) so you can target and report on removal again, then delete it as the final step.

"The Uninstall shim I created isn't detecting the existing install"

The shim's **detection rule** must match the *already-installed* software, not the placeholder you packaged. Point the detection rule at the real app's MSI product code / file+version / registry key so Intune sees the app as present (then absent after the shim's uninstall command runs).

"Supersedence isn't removing the old app"

- Confirm **Uninstall the previous version = Yes** on the *replacement* app's Supersedence tab (Implementation Step 8), and that the replacement is actually deploying to the device (supersedence acts as the replacement installs).
- Supersedence changes can take **up to a day** plus a couple of check-ins to fully evaluate. Give it time before assuming failure.
- Deeply nested supersedence chains (A supersedes B supersedes C) evaluate one hop at a time — don't

skip levels.

"The reports don't match what's on the devices" (reporting gaps)

• Intune app install reports lag device reality by a check-in cycle (and up to several hours for the aggregated install summary). Use `deviceStatuses` (per device, which the scripts read) rather than the summary tile for the freshest picture, and allow a cycle before trusting counts.

• A device that is uninstalled but hasn't checked in still shows its **last** state. This is why the evidence pack (`APPRET-EvidenceExport.ps1`) lists residuals *with last check-in* — a "residual" that hasn't synced in 40 days is an offline device, not a failed uninstall.

Script / access problems

• `Connect-MgGraph` **fails or scopes are denied:** a Global Administrator must grant admin consent for the read scopes in Requirement 2.2. Re-run after consent.

• **US Gov tenant returns 401/empty:** you connected to the wrong cloud. Add `-Environment USGov` (GCC High) or `-Environment USGovDoD` (DoD) so the scripts hit `graph.microsoft.us` / `dod-graph.microsoft.us`. If a specific object type still returns nothing, **verify for GCC High** that the feature is available in your sovereign tenant.

• `APPRET-ArtifactFinder.ps1` **can't read CA policies:** it needs `Policy.Read.All`; enterprise-app and CA sections are skipped without it — grant the scope or review those artefacts manually.

• **Group member** `$count` **shows** `?`**:** the `ConsistencyLevel: eventual` header/advanced-query wasn't honored (older tenant/SDK). The count is informational; membership is still correct.

7. Best Practices — Application Retirement, Offboarding & Decommissioning

Field-proven habits that keep retirements safe, defensible, and repeatable.

Decision discipline

• **Never retire an app nobody will vouch for without a named risk-acceptor.** "No owner" is evidence *for* retirement, but someone senior must sign that they accept the outage risk.

• **Name the trigger explicitly.** The trigger (EOL/EOS, security, redundancy, low utilisation, cost) sets the grace period and approver. A cost-driven retirement with a renewal date is a *deadline*, not a wish.

• **Separate install base from usage.** Intune tells you who *has* the app; it never tells you who

uses it. Get a real usage signal (vendor console, SSO sign-ins, licence logs) before you call something "unused".

Protecting data (the irreversible step)

- **Treat the uninstall as a one-way door.** Always determine per app what the uninstaller destroys, on a test device, before Wave 0.
- **Export before you assign.** Do not create the Uninstall assignment until required data is exported and verified. An assignment can reach a fast-syncing device in minutes.
- **Capture licence/activation state before removal.** Some products deactivate on uninstall; record the seat count and keys first.

Campaign mechanics

- **Understand intent precedence. Required beats Uninstall.** The most common "nothing happened" cause is an old Required assignment still winning. Remove the install assignment; don't just stack an Uninstall on top.
- **Delete the app object LAST.** Deleting it does not uninstall anything and destroys your ability to target and report. It is the final cleanup step, never the first.
- **Use assigned groups, never a dynamic sweep.** A dynamic "all devices with the app" rule can uninstall the whole fleet in one sync. Assigned wave groups keep the blast radius chosen and auditable.
- **Always keep an exclusion group with an expiry in its name.** Exemptions that never expire become permanent shadow deployments.
- **Prefer supersedence for migrations.** When a replacement exists, "Uninstall previous version = Yes" guarantees a device is never left with neither app.
- **Deploy the replacement first, prove it, then remove the old app.** Never remove before the successor is validated on the same devices.

Stragglers and reach

- **Define "done" in the presence of unreachable devices.** Set a completion threshold on *reachable* devices and require every remaining device to be dispositioned (offline / decommissioned / escalated / exempt). A campaign that waits for 100% of a fleet including dead devices never closes.
- **Escalate installer-level failures; don't debug them in the campaign.** A device whose uninstall genuinely errors is a Win32 uninstall failure remediation task — record and hand it off, keep the wave moving.

Cleanup and proof

• **Clean up every artefact, not just the app.** Assignments, wave/exempt groups, detection and remediation scripts, WDAC / App Control rules, Conditional Access references, the enterprise application, and SCCM remnants. Orphaned app-control rules are a genuine security hazard — a stale *allow* widens attack surface; a stale *deny* blocks a future app with the same name.

• **Never delete a shared script or group.** The ImpactAnalyzer and ArtifactFinder flag shared objects — leave them.

• **The evidence pack is the deliverable.** "Is it gone?" must be answerable with data: final status, dispositioned residuals, zero artefacts, data-export record, licence numbers.

• **Monitor for reappearance.** Re-imaged machines, returned-from-offline devices, and lingering SCCM deployments bring apps back. Keep watching for 30–60 days and kill the source, not just the symptom.

Naming and repeatability

• **One retirement code per campaign** (`APPRET-<code>`), used in every object name, so a whole campaign sorts and cleans up together.

• **Reuse the exact object names** from the Implementation Guide verbatim in scripts, testing, and the evidence pack.

• **Keep the decision record as the campaign's front page.** It carries the trigger, evidence, approver, data determination, wave dates, licence numbers, and cleanup checklist — the story of the retirement in one place.

Deliverables Summary

This solution delivers a complete, repeatable process plus tooling for retiring an application from an Intune-managed Windows fleet and proving it is decommissioned.

Documents

Document	What it gives you
Solution Overview (`01-solution-overview.md`)	The retirement pipeline, the five triggers, exactly how Intune's Uninstall intent behaves (and why Required beats it), waves vs big-bang, and the honest limitations (irreversibility, check-in bound, per-device failures out of scope).
Global Requirements (`global-requirements-intune-solutions.md`)	Category-level prerequisites (tenant, Intune, enrollment, Windows baseline, network).

Document	What it gives you
Requirements (`02-requirements.md`)	Solution-specific prerequisites — roles, Graph read scopes, a working uninstall command, a reliable detection rule, usage evidence, a named owner/approver, the data-handling determination, wave groups, and artefact knowledge — each with Description / Why / Verify / Implement.
Implementation Guide (`03-implementation.md`)	The 15-step campaign: name the trigger and decision record, discovery, impact analysis, comms + grace period, wave groups, the DATA checkpoint, point the Uninstall intent (or use supersedence), validate, expand, reclaim licences, handle stragglers, clean up every artefact, produce the evidence pack, monitor. Every object is named and reused.
Testing (`05-testing.md`)	Seven pass/fail gates from data preservation through evidence + closure, plus a full-campaign regression.
Troubleshooting (`06-troubleshooting.md`)	Campaign-level problems: devices not checking in, partial fleet removal, "nothing happened", reappearance, reporting gaps — and when to escalate a per-device failure.
Best Practices (`07-best-practices.md`)	Decision discipline, protecting data, intent precedence, cleanup, straggler reach, and repeatability.
Executive Summary (`executive-summary.md`)	The business case in one page.

PowerShell scripts

All scripts use the Microsoft Graph PowerShell SDK and accept `-Environment USGov` / `USGovDoD` for sovereign clouds. All five are **read-only by default**; the two planning helpers write only files you name, and no script performs an uninstall or a deletion — removal actions are done deliberately in the Intune portal per the Implementation Guide.

Script	Purpose	Access
APPRET-Discovery.ps1	Install base + assignments/intents + dependency/supersedence + per-device install list (evidence exhibit A).	Read-only
APPRET-ImpactAnalyzer.ps1	What breaks if removed: apps that depend on it, supersedence chains, assign groups + member counts, shared detection scripts.	Read-only

Script	Purpose	Access
APPRET-WaveReporter.ps1	Wave `Status` (gate check), `Plan` (split install base into balanced waves), `Stragglers` (classify non-removed devices).	Read-only
APPRET-ArtifactFinder.ps1	Leftover artefacts: app object, groups (SAFE vs SHARED), CA references, enterprise app, remediation scripts.	Read-only
APPRET-EvidenceExport.ps1	Compile the decommission evidence pack (final status, residuals, remaining artefacts) to a timestamped folder.	Read-only

How the pieces fit

1. Confirm **Requirements**; open the **Decision Record** and name the trigger.
2. Run **Discovery** for the install base; obtain a real usage signal.
3. Run **ImpactAnalyzer**; resolve anything that depends on the app.
4. Communicate; set the grace period; build **wave groups** and the exclusion group.
5. **Protect data** (the irreversible checkpoint), then point the **Uninstall** intent at Wave 0

(or configure supersedence for a migration).

1. **Validate** with the testing gates; use **WaveReporter** to expand wave by wave and classify **stragglers**.

1. Reclaim **licences**; use **ArtifactFinder** to clean up every leftover.
2. Run **EvidenceExport** for the proof pack; **monitor** for reappearance, then close.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

APPRET-ArtifactFinder.ps1

Purpose: Answers "what does this app leave behind?" Builds the cleanup checklist for the end of a retirement: the app object + its assignments, the Entra groups used to target it (and whether any OTHER policy/app also uses those groups), Conditional Access policies that name the app's enterprise application, app registrations / enterprise apps matching the name, and proactive- remediation or platform scripts named for the app. Strictly read-only — it reports; you delete by hand (Implementation Step 13) after confirming nothing live still uses each item. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPRET-ArtifactFinder.ps1 -AppName "LegacyTrack Client" -AppCode "APPRET-LGT" US Gov: add -Environment USGov or -Environment USGovDoD Requires the Microsoft Graph PowerShell SDK with DeviceManagementApps.Read.All, DeviceManagementConfiguration.Read.All, Group.Read.All, Application.Read.All, Policy.Read.All. EXPECTED RESULTS Sections listing each candidate artefact and whether it appears SAFE to remove (nothing else references it) or SHARED (leave it). Nothing is deleted. .NOTES Intune Solutions — read-only. Artefact-cleanup planning stage of an application retirement. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AppName, [string]$AppCode, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\APPRET-ArtifactFinder.ps1 -AppName "LegacyTrack Client" -AppCode "APPRET-LGT" US Gov: add -Environment USGov or -Environment USGovDoD Requires the Microsoft Graph PowerShell SDK with DeviceManagementApps.Read.All, DeviceManagementConfiguration.Read.All, Group.Read.All, Application.Read.All, Policy.Read.All. EXPECTED RESULTS Sections listing each candidate artefact and whether it appears SAFE to remove (nothing else references it) or SHARED (leave it). Nothing is deleted. .NOTES Intune Solutions — read-only. Artefact-cleanup planning stage of an application retirement. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AppName, [string]$AppCode, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

APPRET-Discovery.ps1

Purpose: Answers "who still has this app, and how is it assigned?" — the starting point of a retirement campaign. Reports the Intune app object(s) matching a name, every assignment intent and target group, dependency/supersedence links, and the per-device install list (device, primary user, last check-in) so you can plan waves and build evidence exhibit A. Strictly read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPRET-Discovery.ps1 -AppName "LegacyTrack Client" powershell -ExecutionPolicy Bypass -File .\APPRET-Discovery.ps1 -AppName "LegacyTrack Client" -CsvPath .\install-base.csv US Gov: add -Environment USGov (GCC High) or -Environment USGovDoD (DoD) Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A per-app summary (intents, target groups, dependencies, supersedence) and an install-state summary; with -CsvPath, a per-device CSV of every device that has the app. .NOTES Intune Solutions — read-only. Discovery / planning stage of an application retirement. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AppName, [string]$CsvPath, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\APPRET-Discovery.ps1 -AppName "LegacyTrack Client" powershell -ExecutionPolicy Bypass -File .\APPRET-Discovery.ps1 -AppName "LegacyTrack

Client" -CsvPath .\install-base.csv US Gov: add -Environment USGov (GCC High) or -Environment USGovDoD (DoD) Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A per-app summary (intents, target groups, dependencies, supersedence) and an install-state summary; with -CsvPath, a per-device CSV of every device that has the app. .NOTES Intune Solutions — read-only. Discovery / planning stage of an application retirement. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AppName, [string]$CsvPath, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

APPRET-EvidenceExport.ps1

Purpose: Produces the proof-of-decommission pack that answers "is this app genuinely gone?" Gathers the final install/uninstall status, the residual-installed device list with last check-in (so each remaining device can be dispositioned), whether the Intune app object still exists, whether any Entra groups / enterprise application / CA reference for the app remain, and writes it all to a timestamped folder (per-section CSVs + a summary.txt). Strictly read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPRET-EvidenceExport.ps1 -AppName "LegacyTrack Client" -OutFolder .\APPRET-LGT-Evidence US Gov: add -Environment USGov or -Environment USGovDoD Requires the Microsoft Graph PowerShell SDK with DeviceManagementApps.Read.All, Group.Read.All, Application.Read.All. EXPECTED RESULTS A folder containing summary.txt, residual-installed.csv, final-status.csv, and remaining-artifacts.csv — attach with the Decision Record as the evidence pack. .NOTES Intune Solutions — read-only. Evidence stage of an application retirement. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AppName, [string]$OutFolder = ".\APPRET-Evidence", [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\APPRET-EvidenceExport.ps1 -AppName "LegacyTrack Client" -OutFolder .\APPRET-LGT-Evidence US Gov: add -Environment USGov or -Environment USGovDoD Requires the Microsoft Graph PowerShell SDK with DeviceManagementApps.Read.All, Group.Read.All, Application.Read.All. EXPECTED RESULTS A folder containing summary.txt, residual-installed.csv, final-status.csv, and remaining-artifacts.csv — attach with the Decision Record as the evidence pack. .NOTES Intune Solutions — read-only. Evidence stage of an application retirement. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AppName, [string]$OutFolder = ".\APPRET-Evidence", [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

APPRET-ImpactAnalyzer.ps1

Purpose: Answers "what breaks if I remove this app?" Surfaces the blast radius BEFORE you retire an app: other Intune apps that declare this app as a DEPENDENCY (they will fight your uninstall), the supersedence chains this app sits in, the Entra groups used to assign it (with member counts), and Win32 apps that share the same detection/requirement script. Strictly read-only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPRET-ImpactAnalyzer.ps1 -AppName "LegacyTrack Client" US Gov: add -Environment USGov or -Environment USGovDoD Requires the Microsoft Graph PowerShell SDK, DeviceManagementApps.Read.All, Group.Read.All. EXPECTED RESULTS Sections for: apps that DEPEND ON this app (must resolve first), supersedence in/out, assign groups + member counts, and shared detection scripts — each an item for the decision record. .NOTES Intune Solutions — read-only. Impact-analysis stage of an application retirement. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AppName, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\APPRET-ImpactAnalyzer.ps1 -AppName "LegacyTrack Client" US Gov: add -Environment USGov or -Environment USGovDoD Requires the Microsoft Graph PowerShell SDK, DeviceManagementApps.Read.All, Group.Read.All. EXPECTED RESULTS Sections for: apps that DEPEND ON this app (must resolve first), supersedence in/out, assign groups + member counts, and shared detection scripts — each an item for the decision record. .NOTES Intune Solutions — read-only. Impact-analysis stage of an application retirement. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AppName, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

APPRET-WaveReporter.ps1

Purpose: The operational reporter for a running uninstall campaign. Three actions: -Action Status : per-device uninstall/detection status for the app, summarised and (optionally) exported — the wave gate check. (read-only) -Action Plan : split an install-base CSV (from APPRET-Discovery) into balanced wave lists, spread across departments/OU, so waves are representative. (read-only; writes CSVs only if -OutFolder is given) -Action Stragglers : classify devices that have NOT uninstalled — Offline/NeverCheckedIn, StillRequired (an install intent still wins), or UninstallFailed (escalate to Win32 uninstall failure remediation). (read-only) HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPRET-WaveReporter.ps1 -AppName "LegacyTrack Client" -Action Status powershell -ExecutionPolicy Bypass -File .\APPRET-WaveReporter.ps1 -Action Plan -InstallBaseCsv .\install-base.csv -Waves 3 -OutFolder .\waves powershell -ExecutionPolicy Bypass -File .\APPRET-WaveReporter.ps1 -AppName "LegacyTrack Client" -Action Stragglers -StaleDays 14 US Gov: add -Environment USGov or -Environment USGovDoD Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS Status: Installed/Uninstalled/Failed counts + optional CSV. Plan: N wave CSVs. Stragglers: a classified table with each device's disposition for the evidence pack. .NOTES Intune Solutions — read-only reporting/planning. Execution stage of an application retirement. > [CmdletBinding()] param([ValidateSet('Status','Plan','Stragglers')][string]$Action = 'Status', [string]$AppName, [string]$CsvPath, [string]$InstallBaseCsv, [int]$Waves = 3, [int]$StaleDays = 14, [string]$OutFolder, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\APPRET-WaveReporter.ps1 -AppName "LegacyTrack Client" -Action Status powershell -ExecutionPolicy Bypass -File .\APPRET-WaveReporter.ps1 -Action Plan -InstallBaseCsv .\install-base.csv -Waves 3 -OutFolder .\waves powershell -ExecutionPolicy Bypass -File .\APPRET-WaveReporter.ps1 -AppName "LegacyTrack Client" -Action Stragglers -StaleDays 14 US Gov: add -Environment USGov or -Environment USGovDoD Requires the Microsoft Graph PowerShell SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS Status: Installed/Uninstalled/Failed counts + optional CSV. Plan: N wave CSVs. Stragglers: a classified table with each device's disposition for the evidence pack. .NOTES Intune Solutions — read-only reporting/planning. Execution stage of an application retirement. > [CmdletBinding()] param([ValidateSet('Status','Plan','Stragglers')][string]$Action = 'Status', [string]$AppName, [string]$CsvPath, [string]$InstallBaseCsv, [int]$Waves = 3, [int]$StaleDays = 14, [string]$OutFolder, [ValidateSet('Global','USGov','USGovDoD')][string]$Environment = 'Global')